

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине «Вычислительная математика»
Тема: Интерполирование функций

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Приблизить функцию с помощью интерполирования сплайнами. Сравнить результат с исходной функцией и с интерполированием через многочлен Лагранжа.

Теоретический минимум.

Интерполяция сплайнами

Пусть на отрезке $[a, b]$ заданы n узлов интерполяции x_i и известны значения функции $y_i = f(x_i)$ в этих узлах. Требуется построить функцию $\varphi(x)$, которая проходит через все заданные точки, то есть $\varphi(x_i) = y_i$ для всех $i = 0, 1, \dots, n-1$, и позволяет вычислять значения функции в произвольных точках внутри отрезка.

На каждом отрезке между узлами функция представляется кубическим многочленом. На всём отрезке сплайн дважды непрерывно дифференцируем. Форма сплайна на отрезке $[x_i, x_{i+1}]$:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Для обеспечения гладкости сплайна на всём отрезке накладываются следующие условия:

- $S_i(x_i) = y_i$ и $S_i(x_{i+1}) = y_{i+1}$ для каждого i ;
- Непрерывность первой производной $S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$;
- Непрерывность второй производной $S''_i(x_{i+1}) = S''_{i+1}(x_{i+1})$;
- Граничные условия $S''(x_0) = 0$ и $S''(x_{n-1}) = 0$ (естественный сплайн).

В результате подстановки условий получается система линейных уравнений относительно вторых производных $S''(x_i) = c_i$. Эта система имеет трёхдиагональный вид и решается методом прогонки, что даёт вычислительную сложность $O(n)$.

Интерполяция с помощью многочлена Лагранжа

Пусть на отрезке $[a, b]$ заданы n узлов интерполяции x_i и известны значения функции $y_i = f(x_i)$ в этих узлах. Требуется построить многочлен, проходящий через все заданные точки.

Многочлен Лагранжа имеет вид: $L(x) = \sum_{i=0}^{n-1} y_i \cdot l_i(x)$,

где $l_i(x)$ — базисные многочлены Лагранжа, определяемые формулой:

$l_i(x) = \prod_{j=0, j \neq i}^{n-1} \frac{x - x_j}{x_i - x_j}$. Каждый базисный многочлен обладает свойством $l_i(x_i) = 1$ и $l_i(x_j) = 0$, при $j \neq i$.

Выполнение работы.

Создана функция $f(x)$, вычисляющая значение подынтегральной функции $f(x) = \lg(x) - \frac{1}{x}$ в заданной точке. Пользователь вводит количество узлов интерполяции n , после чего на отрезке $[2.0; 4.0]$ равномерно генерируются узлы x_i и вычисляются соответствующие значения $y_i = f(x_i)$. Полученные пары (x_i, y_i) сохраняются в файл `func.txt`. Также вычисляются середины каждого отрезка между узлами и значения функции в этих точках, которые сохраняются в файл `mids.txt` для последующей оценки точности интерполяции.

Реализована функция `lagrange_interpolation()`, которая строит интерполяционный многочлен Лагранжа степени $n-1$, проходящий через все заданные узлы. Вычисляются значения этого многочлена в серединах отрезков между узлами. Результаты сохраняются в файл `lagrange.txt`.

Реализована функция `spline_interpolation()`, которая строит естественный кубический сплайн. На каждом отрезке $[x_i, x_{i+1}]$ сплайн представляется кубическим многочленом. Для определения коэффициентов сплайна решается трёхдиагональная система линейных уравнений относительно вторых производных в узлах. Граничные условия выбраны для естественного сплайна.

Реализована функция `solve_tridiagonal_matrix()`, которая решает трёхдиагональную систему методом прогонки. Этот метод имеет

вычислительную сложность $O(n)$ и является оптимальным для систем такого типа. На каждом шаге прямого хода вычисляются прогоночные коэффициенты, затем выполняется обратный ход для нахождения неизвестных.

Для каждого отрезка интерполяции вычисляются коэффициенты кубического сплайна a_i , b_i , c_i , d_i , которые сохраняются в файл spline.txt. Эти коэффициенты позволяют восстановить значение сплайна в любой точке отрезка.

Тестирование.

На вход программе подавались различные значения количества узлов интерполирования. Результаты полученной сплайн-функции, а также погрешность сплайн-интерполяции и интерполяции Лагранжа относительно исходной функции представлены на рисунках 1-3.

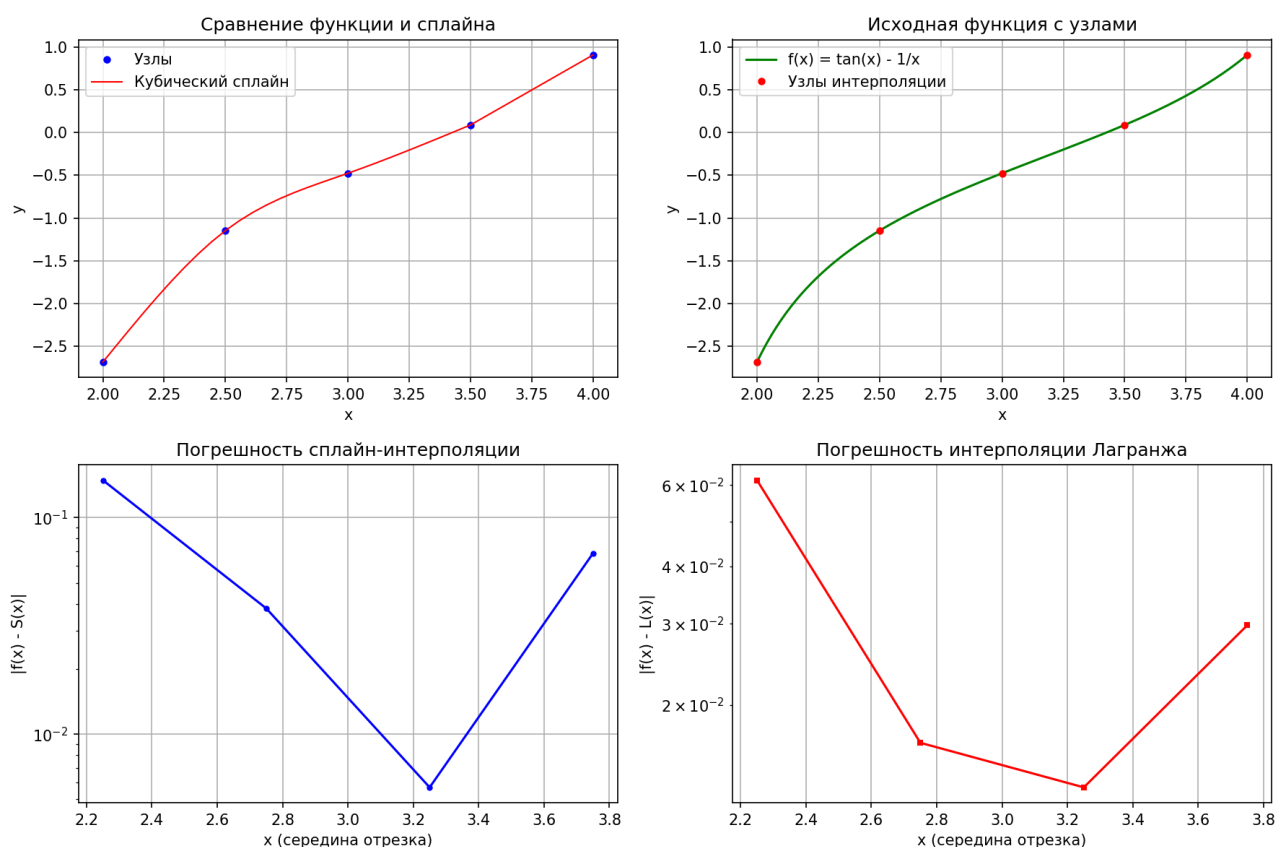


Рисунок 1 — 5 узлов

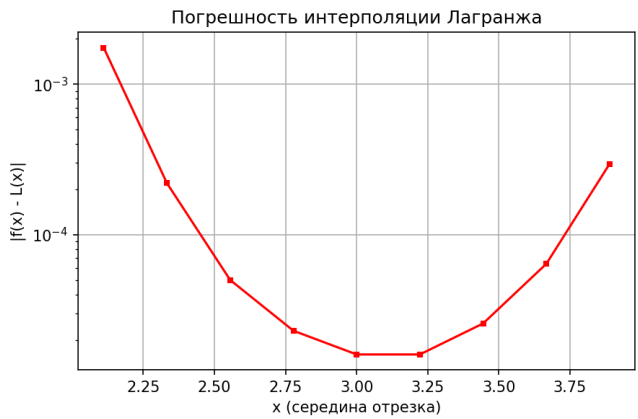
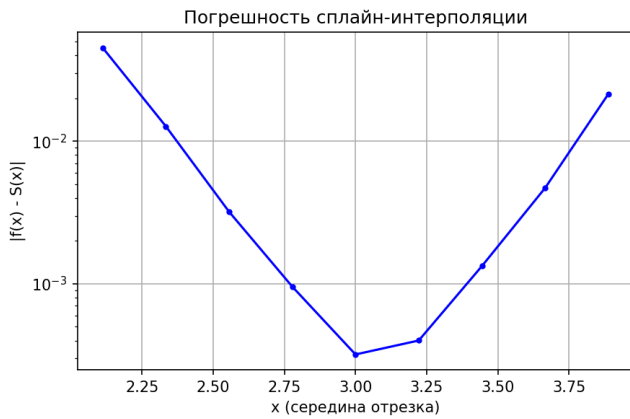
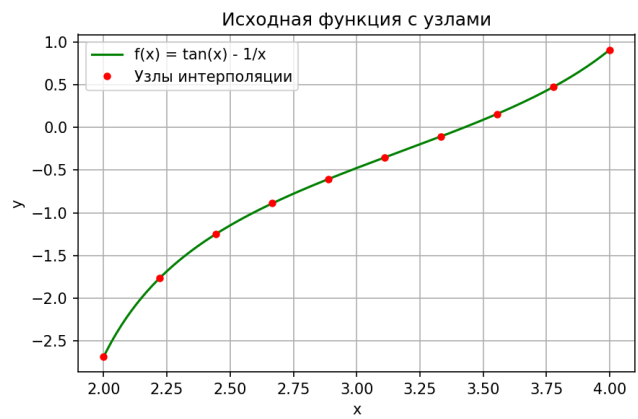
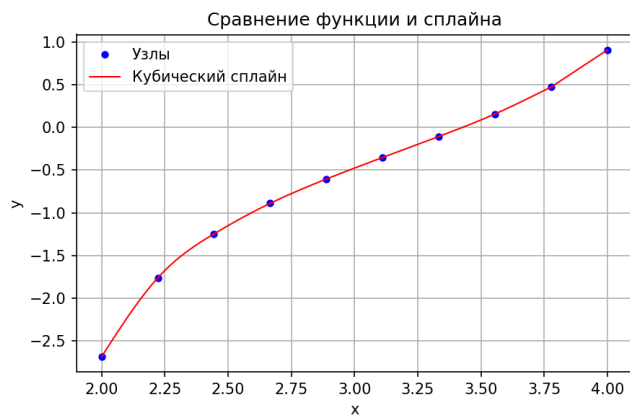


Рисунок 2 — 10 узлов

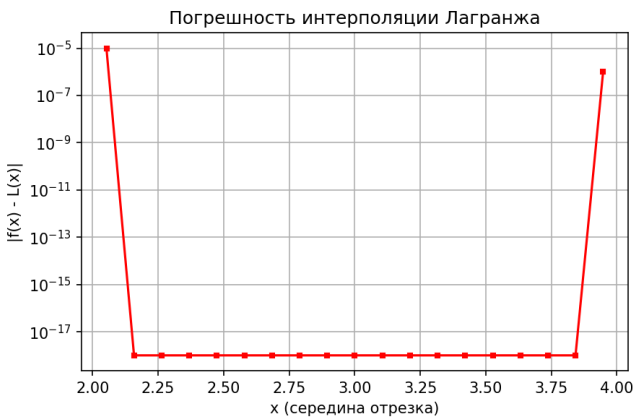
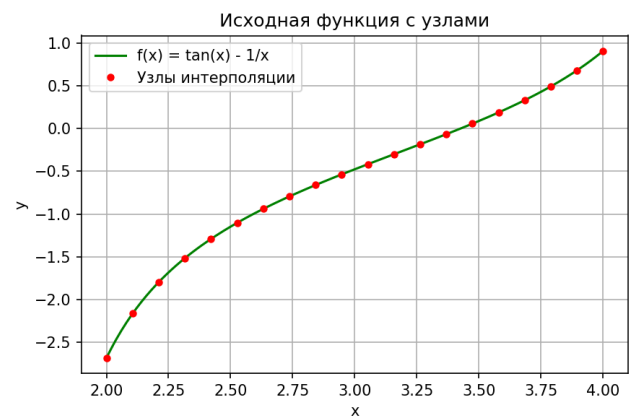
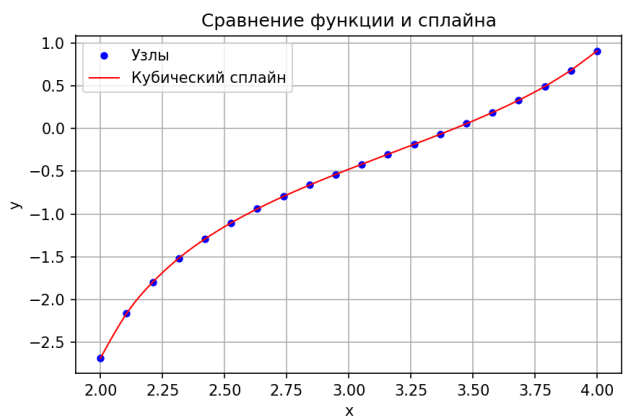


Рисунок 3 — 20 узлов

Вывод.

В результате лабораторной работы получена приближенная функция с помощью интерполяции кубическими сплайнами и многочлена Лагранжа. Оценены погрешности обоих методов относительно исходной функции, проведено тестирование на различных количествах узлов интерполирования.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

Файл `spline_interpolation.cpp`

```
#include <iostream>
#include <cmath>
#include <vector>
#include <fstream>

double f(double x){
    return tan(x) - 1/x;
}

std::vector<double>
solve_tridiagonal_matrix(std::vector<std::vector<double>> matrix, int n){
    int m = n - 2;

    std::vector<double> a(m);
    std::vector<double> b(m);
    std::vector<double> S(n, 0.0);

    a[0] = -matrix[0][1] / matrix[0][0];
    b[0] = matrix[0][m] / matrix[0][0];

    for (int i = 1; i < m-1; ++i){
        double A = matrix[i][i-1];
        double B = matrix[i][i];
        double C = matrix[i][i+1];
        double D = matrix[i][m];

        double denom = A*a[i-1] + B;

        a[i] = -C/denom;
        b[i] = (D - A*b[i-1])/denom;
    }
```

```

    S[m] = b[m-1];

    for (int i = m-2; i >= 0; --i)
        S[i+1] = a[i]*S[i+2] + b[i];

    S[0] = S[n-1] = 0.0;

    return S;
}

void spline_interpolation(std::vector<double> h,
std::vector<double> y, int n, std::ofstream& f){
    int m = n - 2;
    std::vector<std::vector<double>> system_matrix(m,
std::vector<double>(m + 1, 0.0));

    for (int i = 0; i < m; ++i){
        int idx = i + 1;

        double A = h[idx - 1] / 6.0;
        double B = (h[idx - 1] + h[idx]) / 3.0;
        double C = h[idx] / 6.0;
        double D = (y[idx + 1] - y[idx]) / h[idx] - (y[idx] - y[idx
- 1]) / h[idx - 1];

        if (i > 0) system_matrix[i][i - 1] = A;
        system_matrix[i][i] = B;
        if (i < m - 1) system_matrix[i][i + 1] = C;
        system_matrix[i][m] = D;
    }

    std::vector<double> S = solve_tridiagonal_matrix(system_matrix,
n);

    for (int i = 0; i < n-1; ++i){
        double a = y[i];
        double c = S[i] / 2.0;

```



```

        double d = (S[i + 1] - S[i]) / (6.0 * h[i]);
        double b = (y[i + 1] - y[i]) / h[i] - h[i]*(2.0 * S[i] +
S[i+1]) / 6.0;
        f << a << ' ' << b << ' ' << c << ' ' << d << '\n';
    }
}

```

```

void lagrange_interpolation(std::vector<double> x,
std::vector<double> y, std::vector<double> my_x, std::ofstream& f){
    int n = x.size();

    for (int k = 0; k < my_x.size(); ++k){
        double res = 0.0;
        for (int i = 0; i < n; ++i){
            double prod = y[i];
            for (int j = 0; j < n; ++j){
                if (j != i){
                    prod *= (my_x[k] - x[j]) / (x[i] - x[j]);
                }
            }
            res += prod;
        }

        f << res << '\n';
    }
}

```

```

int main(){
    int n;
    std::cout << "Введите количество узлов интерполирования: ";
    std::cin >> n;

    std::ofstream file_spline("spline.txt");
    std::ofstream file_func("func.txt");
    std::ofstream file_middles("mids.txt");
    std::ofstream file_lagrange("lagrange.txt");
}

```

```

double a = 2.0;
double b = 4.0;

std::vector<double> nodes(n);
std::vector<double> arr_y(n);
std::vector<double> arr_h(n-1);
std::vector<double> middles_h(n-1);
std::vector<double> middles_h_y(n-1);

for (int i = 0; i < n; ++i){
    double x = a + i *(b-a) / (n-1);
    nodes[i] = x;
    arr_y[i] = f(x);

    file_func << nodes[i] << ' ' << arr_y[i] << '\n';
}

for (int i = 0; i < n-1; ++i){
    arr_h[i] = nodes[i+1] - nodes[i];
    middles_h[i] = (nodes[i] + nodes[i+1]) / 2.0;
    middles_h_y[i] = f(middles_h[i]);

    file_middles << middles_h[i] << ' ' << middles_h_y[i] << '\n';
}

spline_interpolation(arr_h, arr_y, n, file_spline);    //n-1
уравнений, в каждом 4 неизвестных

lagrange_interpolation(nodes, arr_y, middles_h, file_lagrange);
//Значения многочлена Лагранжа в серединах отрезков

file_spline.close();
file_lagrange.close();
file_func.close();
file_middles.close();

```

```
        return 0;
    }
```

Файл benchmarks.py

```
import matplotlib.pyplot as plt
import numpy as np
from math import pow

f_spline = open("spline.txt")
f_func = open("func.txt")
f_mid = open("mids.txt")
f_lagr = open("lagrange.txt")

mids_x, mids_y = [], []
arr_x, arr_y = [], []
lagr_vals = [float(s) for s in f_lagr]
spline_coeffs = []

for s in f_spline:
    coeffs = list(map(float, s.split(' ')))
    spline_coeffs.append(coeffs)

for s in f_func:
    x, y = list(map(float, s.split(' ')))
    arr_x.append(x)
    arr_y.append(y)

for s in f_mid:
    x, y = list(map(float, s.split(' ')))
    mids_x.append(x)
    mids_y.append(y)

diff_spline, diff_lagr = [], []
spline_x = []
spline_y = []

for i in range(len(mids_x)):
```

```

    spline_val = 0.0
    for j in range(len(spline_coeffs[i])):
        spline_val += spline_coeffs[i][j] * pow(mids_x[i] -
arr_x[i], j)

    diff_spline.append(abs(mids_y[i] - spline_val))
    diff_lagr.append(abs(mids_y[i] - lagr_vals[i]))

    for i in range(len(arr_x) - 1):
        sub_x = np.linspace(arr_x[i], arr_x[i+1], 10)    #10 точек на
отрезке
        for x in sub_x:
            val = 0.0
            for j in range(len(spline_coeffs[i])):
                val += spline_coeffs[i][j] * pow(x - arr_x[i], j)
            spline_x.append(x)
            spline_y.append(val)

    f_spline.close()
    f_func.close()
    f_mid.close()
    f_lagr.close()

    plt.figure(figsize=(12, 8))

    plt.subplot(2, 2, 1)
    plt.plot(arr_x, arr_y, 'bo', label='Узлы', markersize=4) # Только
точки узлов
    plt.plot(spline_x, spline_y, 'r-', label='Кубический сплайн',
linewidth=1)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Сравнение функции и сплайна')
    plt.legend()
    plt.grid(True)

    plt.subplot(2, 2, 2)

```

```

x_fine = np.linspace(min(arr_x), max(arr_x), 200)
y_fine = [np.tan(x) - 1/x for x in x_fine]
plt.plot(x_fine, y_fine, 'g-', label='f(x) = tan(x) - 1/x',
linewidth=1.5)
plt.plot(arr_x, arr_y, 'ro', label='Узлы интерполяции',
markersize=4)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Исходная функция с узлами')
plt.legend()
plt.grid(True)

plt.subplot(2, 2, 3)
plt.semilogy(mids_x, diff_spline, 'b-', marker='o', markersize=3)
plt.xlabel('x (середина отрезка)')
plt.ylabel('|f(x) - S(x)|')
plt.title('Погрешность сплайн-интерполяции')
plt.grid(True)

plt.subplot(2, 2, 4)
plt.semilogy(mids_x, [d + 1e-18 for d in diff_lagr], 'r-',
marker='s', markersize=3)
plt.xlabel('x (середина отрезка)')
plt.ylabel('|f(x) - L(x)|')
plt.title('Погрешность интерполяции Лагранжа')
plt.grid(True)

plt.tight_layout()
plt.savefig('plot.png', dpi=150, bbox_inches='tight')

```